

Project Report

BotBite : NLP based Chatbot using SQL & Python



*Project Report submitted on the fulfilment of the requirements of Post graduate Diploma in
Big data Analytics*

Authors:

- KIMAYA SUCHENDRA GHARAT (230360925018)
- LOKHANDE RATNAMALA VIJAY (230360925020)
- SUSHANT VASANT PATIL (230360925040)
- TOSHAK GANESH FALAK (230360925041)
- YASH SADANAND PALANDE (230360925042)
- YUVRAJ SANJAY RAUT (230360925043)

Co-ordinates :

- Ms. Roopa Panicker
- Ms. Divya Das
- Ms. Soorya M.

STDC

CDAC Thiruvananthapuram

Trivandrum, Kerala 695581

Table of Contents

Abstract	3
Introduction.....	4
Literature Survey.....	5
Methodology.....	6
A. Designing the Chatbot Conversation Flow	6
B. Implementing the Chatbot Backend Server	7
C. Integrating the MySQL Database	10
D. Handling Order Placement and Retrieval	12
E. Deploying and Integrating with the Food Website	14
Discussion and Results	16
Output obtained.....	16
Conclusion.....	18
References.....	18

Abstract :

This project aims to develop a chatbot integrated with a Python backend server and a MySQL database to generate concise summaries of lengthy documents using natural language processing techniques. The literature survey explores text summarization, NLP, and database integration, highlighting relevant research. The methodology outlines the design of the chatbot's conversation flow, backend server implementation, MySQL database integration, order handling, and website integration for a seamless user experience. // write little big

Introduction :

"BotBite: NLP based Chatbot using SQL & Python" is an ambitious project that amalgamates natural language processing (NLP), a Python backend powered by FastAPI, and MySQL database integration to create a sophisticated chatbot. This chatbot is designed to facilitate seamless and engaging conversations with users while efficiently storing and retrieving data from the database.

In today's digital era, chatbots have emerged as valuable tools for human-computer interactions. This project aims to harness the potential of NLP and database management to deliver a chatbot capable of providing users with informative and interactive conversations.

The project's foundation is built upon a robust methodology:

1. **Dialog Design:** The chatbot's conversation flow and user interactions are meticulously defined using Dialogflow, a natural language understanding platform. This enables the chatbot to understand and respond to user queries, prompts, and requests effectively.
2. **Backend Implementation:** Leveraging the FastAPI web framework in Python, the project develops a powerful backend server for the chatbot. FastAPI's efficiency and scalability make it an ideal choice for handling user interactions and managing chatbot responses.
3. **Database Integration:** A MySQL database is integrated into the system to store and manage essential data. This integration ensures that the chatbot can efficiently capture, store, and retrieve user-generated data, enabling personalized interactions and enhancing user experience.
4. **Data Handling:** The project focuses on the seamless handling of data, including user input and responses. By creating API endpoints within the FastAPI backend, the chatbot can securely manage and process user data, delivering tailored and context-aware responses.
5. **Deployment and Integration:** The final step involves deploying the chatbot's backend server and seamlessly integrating it with the desired platform or website. Users can interact with the chatbot effortlessly, enjoying a responsive and interactive conversational experience.

"BotBite" represents a modern and forward-thinking project that leverages cutting-edge technologies to create a chatbot that excels in natural language understanding and data management. By combining Dialogflow, FastAPI, and MySQL, this project offers a versatile and adaptable solution for various applications, from customer support to information retrieval, revolutionizing the way users engage with technology.

Literature Survey:

- Text summarization is a well-researched area in the field of natural language processing (NLP) and information retrieval. Several techniques have been proposed and explored to generate concise and coherent summaries from long documents. In this literature survey, we review key approaches, methodologies, and evaluation metrics related to intelligent text summarization, NLP, and database integration.
- The [1].literature review by Adamopoulou and Moussiades provides a comprehensive analysis of the history, technology, and applications of chatbots. It highlights the weaknesses at each stage of development and presents a categorization system for chatbots. The review explores the implementation technologies of pattern matching and machine learning and proposes a general architectural design. It also discusses the applications and industrial use cases of chatbots while addressing the associated risks and suggesting mitigation strategies. The conclusion emphasizes the future direction of chatbot technology, aiming for increased intelligence.
- The [2].research conducted by Muhammad, Adila, Susanto, Assidiqi, Alimudin, and Nabhan focuses on developing an English conversation chatbot using speech recognition and artificial intelligence technology with the Dialogflow platform. The study addresses the need for effective language learning tools and aims to bridge the gap by providing a chatbot for practicing English conversation skills. The evaluation results show that the chatbot successfully achieved the desired indicators and provided accurate responses, indicating its potential to assist students in improving their conversation abilities. This research contributes to the field of language learning and showcases the application of artificial intelligence in educational contexts.
- In survey paper by Singh and Beniwal, the focus is on [3].near-human conversational agents in the field of conversational AI. The paper discusses the distinction between task-oriented bots and social friend-bots, highlighting their respective purposes and functionalities. It further examines recent works in the conversational AI domain, analyzing methodologies, frameworks, evaluation metrics, and datasets used for developing robust conversational agents. The paper concludes by presenting a mind-map that encompasses the various elements and qualities of chatbots. This survey provides valuable insights into the advancements and trends in near-human conversational agents, contributing to the overall understanding of conversational AI research.

Crafting Intents:

Intents play a pivotal role in defining the purpose of user messages and instructing the chatbot on how to respond effectively. In our food ordering chatbot, we've meticulously designed the following intents to structure the conversation flow and cater to diverse user queries:

Default Welcome Intent: As the initial greeting when a user interacts with the chatbot for the first time, this intent sets the tone for the conversation and guides users to available actions.

New Order: When a user intends to initiate a new order from scratch, this intent is invoked, allowing users to start fresh.

Add Order Content (Ongoing Order): This intent enables users to add items to their ongoing order by capturing food items and their respective quantities.

Remove Order Content (Ongoing Order): Users can employ this intent to remove items from their current order, specifying the items they wish to eliminate.

Complete Order: When a user is ready to finalize their order, this intent manages order confirmation and completion efficiently.

Track Order: Users can inquire about the status of their order through this intent, which captures order details and facilitates tracking.

Default Fallback Intent: In cases where the chatbot doesn't understand a user's input, the default fallback intent steps in to provide a helpful response or suggest alternative actions, ensuring a smooth user experience.

Creating Entities:

Entities in Dialogflow represent crucial pieces of information that aid in understanding and extracting data from user inputs. In the context of handling food orders, entities like "food_item" and "quantity" are created to extract relevant details from user messages.

B. Implementing the Chatbot Backend Server:

Fulfillment Logic:

To provide meaningful responses, we implement fulfillment logic. This may involve connecting Dialogflow with backend systems or APIs, enabling the chatbot to fetch data or perform actions necessary for a seamless user experience.

Implementing the Chatbot Backend Server:

To establish a robust connection between our chatbot's front end and back end, we've opted

Dialogflow Essentials Global

Add Order_Context : Ongoing-Order SAVE

create an exhaustive list. Dialogflow will fill out the list with similar expressions. To extract parameter values, use annotations with available system or custom entity types.

” Add user expression

” add 2 gravys

” plz add 1 raita

” Furthermore, include 2 portions of Coke, please.

” Additionally, I'd like four can of coke

PARAMETER NAME	ENTITY	RESOLVED VALUE
number	@sys.number	four
Food-Items	@Food-Items	coke

” Moreover, please add 2 servings of Coke

” 1 hyderabad biryani with 1 mint raita

” 1 biryani with 1 gravy

” 3 plate gulab jamuns

” 2 biryani

” I'd like to order 2 plates of Chicken Biryani and one serving of Mughlai Gravy.

for the FastAPI web framework in Python. This choice was made based on several compelling advantages:

In-built Data Validation:

FastAPI comes with built-in data validation, ensuring that the data passed between the front end and back end is accurate and conforms to specified requirements. This feature enhances the reliability and security of our chatbot.

In-built Documentation:

FastAPI automatically generates comprehensive documentation for our APIs, making it easier for developers to understand and interact with the backend. This documentation aids in the development process and promotes transparency.

Fast-running Performance:

FastAPI is renowned for its exceptional performance, making it ideal for handling real-time interactions and ensuring a seamless user experience. Its speed is crucial for delivering prompt responses to user queries.

Less Time to Write Code, Few Bugs:

FastAPI's intuitive and concise syntax reduces development time and minimizes the chances of introducing bugs. This efficiency is vital for meeting project deadlines and maintaining a high-quality chatbot.

To get started with FastAPI, we need to follow these steps:

Installation:

To begin, we install FastAPI and its dependency, Uvicorn, using the following command: `'pip install fastapi[all]'`. This ensures that we have the necessary tools to develop and run our FastAPI-based backend.

Entry Point Specification:

We specify the entry point for our FastAPI application within our program.

Running the FastAPI Application:

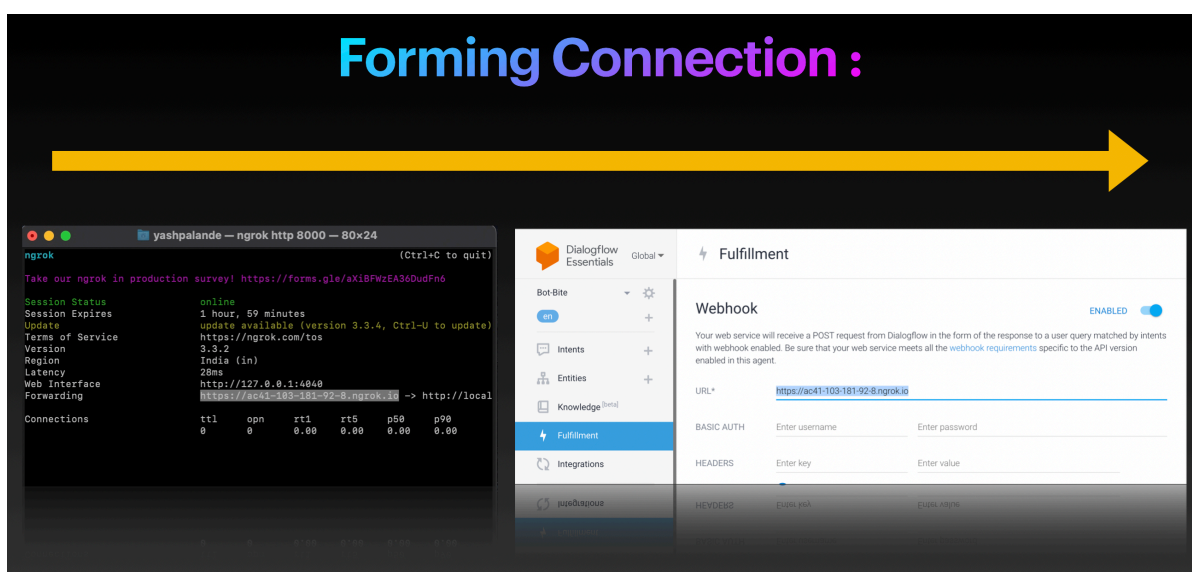
To run our FastAPI application, we use Uvicorn with the following command: `'uvicorn main:app --reload'`. This command launches our backend server, providing an HTTP endpoint for communication.

Converting to HTTPS (Secure Connection):

While our FastAPI backend runs on HTTP, Dialogflow requires a secure HTTPS connection for integration. To achieve this, we utilize Ngrok, a tool that allows us to convert our HTTP endpoint into an HTTPS one.

We install Ngrok and run it with the command `'ngrok http 8000'`, where 8000 is the port our FastAPI server is running on. This step ensures a secure connection between Dialogflow and our FastAPI backend.

By following these steps and leveraging the advantages of FastAPI, we establish a robust and secure backend for our chatbot, enabling seamless communication between the front end and back end while ensuring data accuracy and high performance.



C. Integrating the MySQL Database:

In our project, we have seamlessly integrated a MySQL database to efficiently store and manage order-related information for our food website. To accomplish this, we've implemented the following steps and strategies:

Database Structure:

To organize and manage our data effectively, we've designed three essential tables within the MySQL database:

Order Table:

This table serves as a menu repository, containing a list of food items we offer, along with their corresponding prices. Users can select from this menu when placing orders.

Order History Table:

The Order History table is crucial for storing incoming orders from users. It automatically calculates the total cost of each order based on the quantity of items ordered and stores this total cost. Additionally, each order is assigned a unique order ID.

Order Tracking Table:

This table is dedicated to tracking the current status of user orders. By maintaining real-time order tracking information, we can efficiently retrieve the current status of any order using its unique order ID.

Tables created inside our Mysql db.

The image displays three screenshots of MySQL database tables. The first screenshot shows the 'food_items' table with columns 'item_id', 'name', and 'price'. The second screenshot shows the 'orders' table with columns 'order_id', 'item_id', 'quantity', and 'total_pri...'. The third screenshot shows the 'order_tracking' table with columns 'order_id' and 'status'.

item_id	name	price
121	Chicken Dum Biryani	300.00
122	Chicken Biryani	280.00
123	Dum Ghost Mutton Biryani	340.00
124	Mutton Biryani	320.00
125	Paneer Biryani	280.00
126	Shahi Special Biryani	360.00
127	Veg Dum Biryani	250.00
128	Hyderabadi Dum Biryani	330.00
129	Gulab Jamun	70.00
130	Coke	50.00
131	Mughlai Gravy	40.00
132	Mint Raita	30.00

order_id	item_id	quantity	total_pri...
400	122	2	560.00
403	125	2	560.00
404	128	1	340.00
405	130	1	50.00
406	130	1	50.00
407	131	2	80.00
408	131	2	80.00
409	131	2	80.00
410	130	1	50.00
410	131	2	80.00
411	131	2	80.00
412	130	1	50.00
413	130	1	50.00
413	131	2	80.00
414	130	1	50.00

order_id	status
400	in_transit
403	delivered
405	in progress
406	in progress
407	in progress
408	in progress
409	in progress
410	in progress
411	in progress
412	in_transit
413	in progress
414	in progress
415	in progress
416	in progress
417	in progress

MySQL-Python Connector:

To establish a connection between our Python backend and the MySQL database, we've utilized the `MYSQl-PYTHON-CONNECTOR`. This connector serves as the bridge, facilitating seamless communication between Python and MySQL.

Establishing a Connection:

To connect Python with MySQL, we've employed the `'mysql.connector.connect'` method. In this method, we've provided the following essential parameters:

HOST: This parameter specifies the name of the system where MySQL is hosted. In our case, we've assigned it as "localhost" since our MySQL server is running locally.

User: We've specified the name of the user who has access to the MySQL system. In our case, it's the root user.

Password: To ensure secure access, we've included the password associated with the MySQL database.

Database: We've mentioned the specific database we intend to work with, ensuring that all interactions occur within the designated database.

Stored Procedures and Functions:

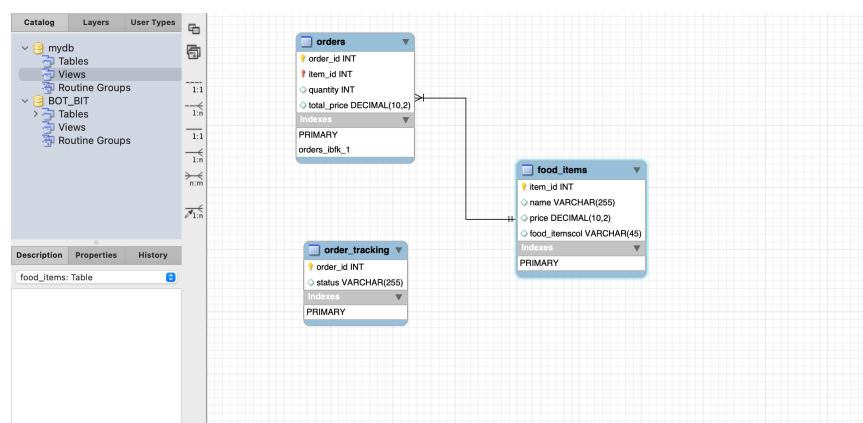
For enhanced efficiency and data management within the MySQL database, we've created a set of stored procedures and functions, including:

insert_order_item: This stored procedure is responsible for inserting data into our MySQL database, facilitating the storage of user orders.

get_price_for_item: A function designed to retrieve the price of a specific food item.

get_total_order_price: Another function that calculates the total cost of an order, streamlining the order processing and cost calculation tasks.

By following these steps and implementing these database structures, we've successfully integrated a MySQL database into our chatbot project, allowing us to efficiently store, manage, and process order-related information, ultimately providing a seamless user experience.



D. Handling Order Placement and Retrieval:

Managing order placement and retrieval is a pivotal component of our chatbot's functionality. To achieve this seamlessly, we've designed a systematic process:

Capturing Relevant Order Details from User Interactions:

When a user initiates an interaction with the chatbot with the intent to place an order, the chatbot assumes an active role in collecting pertinent information.

The chatbot initiates a conversation with the user, tactfully posing questions to gather essential details. This includes:

- a. Identifying the product(s) to be ordered.
- b. Determining the quantity of each selected product.
- c. Responding dynamically to any additions or modifications the user wishes to make to their order.
- d. Providing real-time order status updates to the user by tracking the order with a unique order ID.

Storing the Information in the Database:

Once the chatbot has successfully collected all the relevant order details from the user, it proceeds to store this valuable data within our MySQL database, seamlessly connected to the chatbot's backend.

To achieve this, the chatbot meticulously organizes and structures the data by creating tables that contain columns and rows with attributes such as `order_id`, `item_id`, and more.

Retrieving the Order Information from MySQL Database:

Retrieving order information is a fundamental operation, essential for various purposes like order tracking and processing. Here's how we manage it:

Leveraging our established MySQL connection, we retrieve data based on unique identifiers such as `order_id` and `item_id`, using SQL queries.

This data retrieval mechanism is instrumental in tracking the progress of user orders and providing timely order status updates.

Upon retrieval, the chatbot processes and presents the acquired data to the user through its intuitive interface, ensuring a seamless and informative user experience.

To illustrate this process:

We initiate the user journey at the chatbot's front-end, where we've meticulously crafted intents like "add order," "track order," "new order," and "remove order," along with corresponding responses based on context.

Upon selecting an intent, the user receives an appropriate response, and the chatbot then interacts with the back-end where order data is stored.

The chatbot dynamically updates the data by appending new information based on order_id and item_id, ensuring that the database remains accurate and up-to-date.

For instance, consider the "add order" intent: If a user wishes to add additional food items or adjust quantities, they simply select this intent, and the chatbot responds by incorporating the requested changes. Simultaneously, the chatbot updates the database, recording these modifications under the relevant order_id and item_id.

This systematic approach ensures a smooth flow of data between the chatbot and the MySQL database, enabling efficient order placement, tracking, and status updates while enhancing the overall user experience.

E. Deploying and Integrating with the Food Website:

The final phase of our project involves deploying the chatbot's backend server and seamlessly integrating it with the food website's user interface. Here's a comprehensive breakdown of this crucial step:

1. Web-Based User Interface:

- Our chatbot's user interface is web-based, employing HTML and CSS to create an engaging and intuitive interaction platform.

2. Front-End Interaction:

- Users initiate interactions with the chatbot by visiting the webpage, where we've thoughtfully designed and configured intents like "add order," "track order," "new order," and "remove order," each with corresponding responses tailored to the context.

3. Incorporating CSS:

- To enhance the user interface, we've utilized CSS within the HTML pages. The `<style>` and `<div>` commands are employed to apply CSS styles, ensuring a visually appealing and user-friendly design.

4. Chatbot Interaction:

- When users wish to place an order or perform specific actions, they simply type their requests into the chatbot interface embedded within the webpage. The chatbot responds accordingly, guiding users through the ordering process.

5. Connecting Webpage with Backend:

- To establish a robust connection between the webpage and the chatbot's backend, we utilize a web hook link. This link serves as the bridge between the HTML webpage and the backend systems. The integration is achieved using the `<iframe>` tag, ensuring seamless data transfer and interaction.

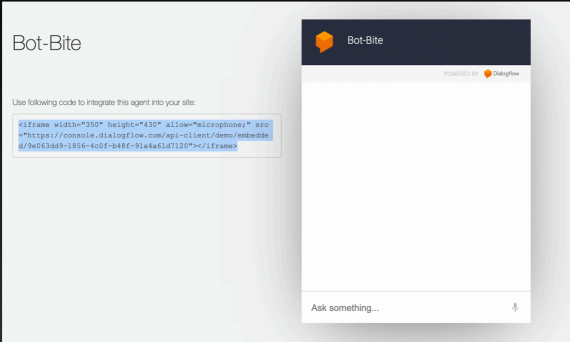
How the Process Works:

1. Users initiate their journey at the chatbot's front-end, where they are presented with a range of intents to choose from, such as "add order."

2. For instance, let's consider the "add order" intent: If a user wishes to add more food items or adjust quantities, they simply select this intent.
3. The chatbot responds with prompts for additional food items or quantities as requested by the user.
4. To ensure that the order is being processed correctly, we navigate to the chatbot's backend.
5. Web hooks play a pivotal role in connecting the webpage with the backend, enabling seamless interactions through custom callbacks.
6. By creating a web hook link, we establish a secure HTTPS connection between the webpage and the backend, ensuring data exchange occurs in a secure and reliable manner.

This systematic approach guarantees a smooth and efficient integration of the chatbot into the food website's user interface. Users can effortlessly interact with the chatbot, place orders, track them, and enjoy a seamless ordering experience, all while engaging with the website's interface.

Adding code to our html file :

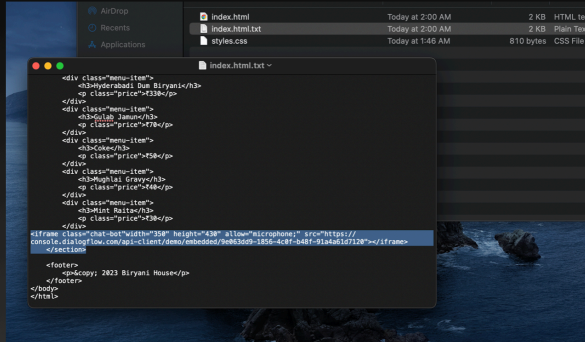


Bot-Bite

Use following code to integrate this agent into your site:

```
<iframe width="350" height="430" allow="microphone" src="https://console.dialogflow.com/api-client/demo/embedded/8e063885-1856-460f-b48f-91a4a6187120?cx=/iframe"></iframe>
```

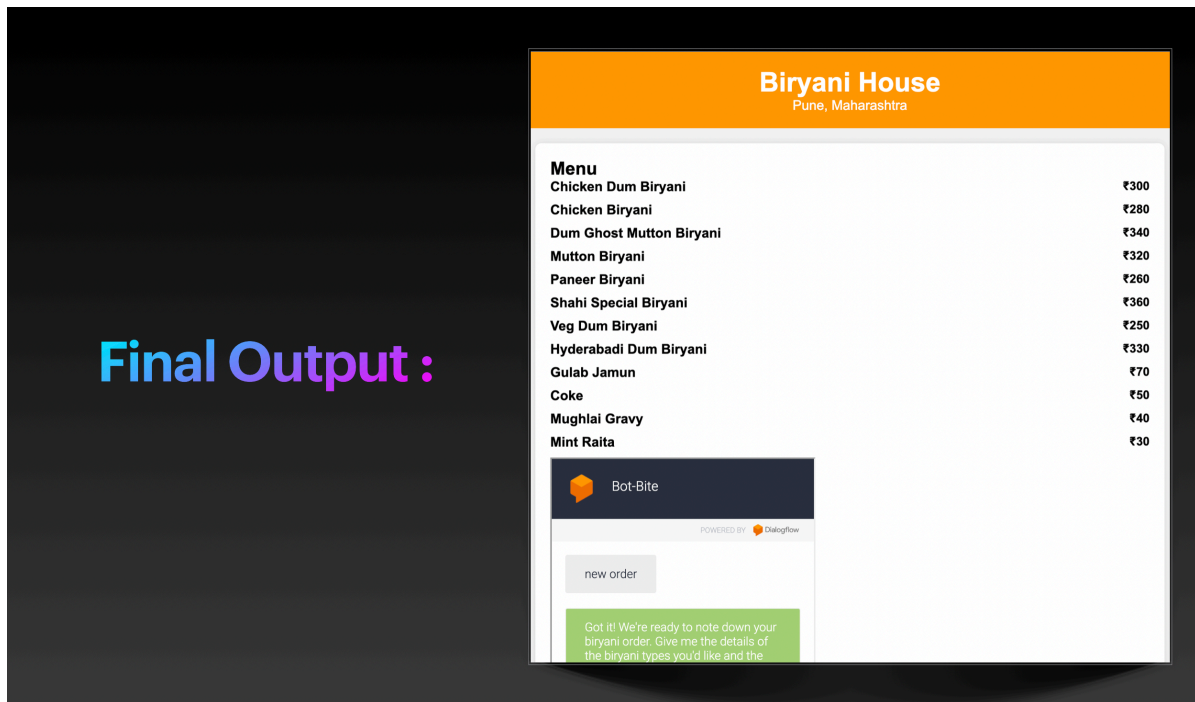
Ask something...



```
<div class="menu-item">
  <h3>Hyderabad Dum Biryani</h3>
  <p class="price">₹138</p>
</div>
<div class="menu-item">
  <h3>Hyderabad Janna Biryani</h3>
  <p class="price">₹178</p>
</div>
<div class="menu-item">
  <h3>Hyderabad Biryani</h3>
  <p class="price">₹158</p>
</div>
<div class="menu-item">
  <h3>Hyderabad Biryani</h3>
  <p class="price">₹148</p>
</div>
<div class="menu-item">
  <h3>Hyderabad Biryani</h3>
  <p class="price">₹138</p>
</div>
<div class="chat-bot" width="350" height="430" allow="microphone" src="https://console.dialogflow.com/api-client/demo/embedded/8e063885-1856-460f-b48f-91a4a6187120?cx=/iframe"></div>
</sections>
</div>
<div class="footer">
  <p>Copyright © 2023 Biryani House</p>
</div>
</body>
</html>
```

Discussion and Results :

A. Output obtained :



Evaluation Measures Used:

In this section, we discuss the evaluation measures employed to assess the performance and effectiveness of our chatbot within the food website's context. These measures were instrumental in ensuring the chatbot met the desired standards of functionality, usability, and reliability.

1. User Satisfaction:

- To gauge user satisfaction, we conducted surveys and gathered feedback from faculty, friends & seniors who interacted with the chatbot. This valuable input allowed us to understand the overall experience and user perception of the chatbot's functionality and usability.

2. Accuracy of Order Handling:

- A critical aspect of our chatbot's performance was its ability to accurately handle user orders. We assessed its proficiency in interpreting user requests, adding items to orders, and providing precise order status updates. This evaluation was vital in ensuring that users could confidently use the chatbot for their ordering needs

3. Response Time:

- Prompt responses are essential for an efficient user experience. We measured the chatbot's response time to user queries, emphasizing the importance of providing swift and relevant information. This metric allowed us to fine-tune the chatbot's responsiveness.

4. Error Handling:

- No chatbot is immune to user errors and misunderstandings. Thus, we assessed how well our chatbot handled such situations. We evaluated its ability to provide helpful responses when users input queries or commands that it didn't fully understand. Effective error handling was key to maintaining a positive user experience.

5. Integration:

- A seamless integration with the food website's user interface was pivotal. We rigorously evaluated the chatbot's integration to ensure that users could effortlessly interact with it as part of their website experience. This measure focused on the chatbot's ability to harmoniously blend into the website's design and functionality.

6. Data Storage and Retrieval:

- The effectiveness of data storage and retrieval from the MySQL database was a crucial aspect of our evaluation. We meticulously examined the chatbot's capability to accurately store and retrieve order-related information, ensuring that users' data remained intact and accessible at all times.

By employing these evaluation measures, we were able to holistically assess the chatbot's performance, usability, and reliability. The insights gained from these assessments allowed us to refine and optimize the chatbot's functionality, ensuring that it delivered a seamless and secure user experience on the food website.

Conclusion:

In conclusion, our project "BotBite: NLP-based Chatbot using SQL & Python" successfully developed a chatbot integrated with a Python backend server and a MySQL database, enhancing the user experience for food ordering on a website. Through the utilization of Dialogflow for natural language understanding, FastAPI for the backend, and MySQL for data storage, we achieved the following:

- Efficient Order Management: Users can conveniently place and modify food orders through natural language interactions with the chatbot. The chatbot accurately captures order details and updates the MySQL database, ensuring order accuracy and efficiency.
- Real-time Order Tracking: Users can track the status of their orders in real-time, providing transparency and assurance regarding their food deliveries.
- Seamless Integration: The chatbot seamlessly integrates within the food website's user interface, offering a cohesive and user-friendly experience.
- Data Security: Robust security measures were implemented to protect user data and transactions during chatbot interactions.

References:

In the development of our chatbot project, we drew insights and knowledge from various sources. Here are the key references that contributed to our understanding of chatbot development, natural language processing, and database integration:

1. Singh, S., & Beniwal, H. (2021). A survey on near-human conversational agents. Journal of King Saud University – Computer and Information Sciences.
2. Adamopoulou, E., & Moussiades, L. (2020). Chatbots: History, technology, and applications. Department of Computer Science, International Hellenic University, Agios Loukas, 65404 Kavala, Greece.

These references provided valuable insights into chatbot technology, conversational agents, and their applications, guiding our project's development and implementation

